PharmAssist DB Performance and Scalability

Author: Justin Hunter
Date: 2026-04-08
Audience: Engineering leadership, the MFC DBA team, the iA DBA team, the iA development teams, and operations
Status: Active

---

1. Purpose

I am writing this charter to set out the program of work that the MFC DBA team and I are leading to relieve current performance pressure on the PharmAssist SQL Server databases, repair and modernize the maintenance procedures that run against them, and put the architecture on a footing that will scale gracefully as we add MFC sites and accumulate data over the next several years.

The program brings together three streams of work that share a common evidence base and a common methodology. It is a multi-team effort. The MFC DBA team and I produce the analyses, the refactor recommendations, the diagnostic toolkit, the pattern reference library, and the recommendation backlog. The iA DBA team and the iA software development teams own the review, validation, non-production testing, and production deployment of any change to the database. None of the work product becomes a production change without flowing through that pipeline.

That structure is normal for a program of this scope, and the iA team's emphasis on a uniform production codebase across all instances is a legitimate engineering concern. It does, however, set the rate at which the program can convert evidence into deployed change. The charter and the companion plans are written with that rate in mind. The deliverable from our side of the program is a documented, evidence-backed backlog of recommendations. The deliverable from the iA side is the production deployment of those recommendations, on a cadence that they manage.

This document ties the streams together so that engineering, operations, and leadership can read a single document and understand the direction, the immediate remediation work, the longer arc commitments, and the working relationship between the teams that will deliver them.

---

2. Background

PharmAssist runs at fourteen to fifteen MFC sites today. Each site operates an instance of the same OLTP schema. The application supports the full pharmacy fulfillment lifecycle: order intake, prescription processing, robotic dispensing, verification, sortation, and shipment.

Two patterns are now visible across the fleet that, taken together, drive most of our database pain.

The first pattern is that a relatively small number of stored procedures and ad hoc queries are doing dramatically more work per execution than they need to. The cross-MFC Query Store capture from May 7, 2026 shows individual procedures generating hundreds of billions of logical reads per month at fleet scale. As an example, `lsp_ShpGetOrdersForTopReadyToShipGroup` is producing roughly 950 billion logical reads per month across 33 million executions, and `lsp_PmssIwebGetTopQueuedTrx` is producing roughly 278 trillion logical reads per month against a query that should cost single-digit reads per call. These are not inherently expensive operations. They have become expensive because of structural anti-patterns in the SQL and gaps in index coverage.

The second pattern is that the highest-volume tables continue to grow. The active working set on most of these tables is roughly the most recent sixty days, but the tables themselves carry hundreds of millions of historical rows. The nightly maintenance procedures responsible for purging that history are doing so through iterative block deletes that work but become slower as the tables grow. A small number of these procedures contain bugs that have prevented them from running at all, and several lack the safety parameters that protect the transaction log on first activation against a backlog.

Both patterns are getting worse with every site we add and every quarter we accumulate. Neither is a hardware problem. Both are addressable through a combination of expert level refactoring, operational hygiene, and a deliberate move to table partitioning on the high-volume tables.

A third pattern, observed in production over the same window, is that the procedures on our optimization list are showing up in the operational record as cancellations and as performance failures. The conditions that motivated putting the procedures on the list in the first place are now producing visible operational events. The optimization work is therefore not a hypothetical exercise. It is responding to incidents that are already happening.

---

3. Team Structure and the Deployment Pipeline

The program has three working parties.

The MFC DBA team and I sit on the analysis and recommendation side. We pull the cross-MFC Query Store data, run the diagnostic toolkit at each site, identify the procedures and statements that warrant a deep dive, write the per-procedure analyses, draft the refactored bodies, generate the supporting evidence, and publish the recommendations into the backlog. We are also the authors of the diagnostic toolkit, the refactor workflow document, the masterclass library of pattern notes, and these planning documents.

The iA DBA team owns the review of any database change before it is permitted to flow further. They are the first gate for a refactor leaving our hands. Their review covers correctness, alignment with the iA team's standards for the production codebase, and consistency with what is deployed across the rest of the iA-managed estate.

The iA software development teams own the second gate. After the iA DBA review, the proposed change goes to the iA developers responsible for the affected functional area for review and validation. They confirm that the refactor preserves application contract, that there are no behavioral side effects in the calling code paths, and that the change is consistent with how that area of the codebase is intended to evolve.

After both reviews complete, the change moves into non-production environments for testing. The duration of that test window is not fixed. It varies by the risk profile of the change and by the iA team's bandwidth at the time. Once the iA team is satisfied with non-production behavior, the change is scheduled into a production release.

Production deployment is not the end of the pipeline. Once a change is rolled to the MFCs, the MFC DBA team and I monitor the behavior of the procedure at each site for a defined window, comparing the post-deployment cost picture against the pre-deployment baseline at the same site. The cross-MFC Query Store evidence already documents wide per-site variance on the same statement (the analysis for `lsp_ShpGetOrdersForTopReadyToShipGroup` shows a 37x spread in reads per execution across sites, and `lsp_PmssIwebGetTopQueuedTrx` shows a 70,000x spread). A single refactored body cannot be assumed to produce uniform improvement across that variance. A refactor that improves the median sites but degrades one or two outlier sites is not a successful refactor at the program level. Post-deployment monitoring is what catches that condition before it becomes an operational event.

This pipeline is an honest description of how the work is delivered today, not a complaint. The iA team's preference for a uniform production codebase across all instances is a real engineering value, and the multi-stage review reflects that. It also means that the rate at which our analyses convert into deployed change is set by the iA team's capacity, not by ours. The corresponding tension on the post-deployment side is that uniform code interacting with non-uniform data shape can produce non-uniform results. Where that happens, root cause analysis is required, and one of three resolutions has to follow: an explanation that grounds the variance in data shape rather than code, a refinement of the deployed code (a hint, a parameter, or a data-

aware branch) that addresses the outlier without breaking the median, or an acknowledged exception at the affected site. The documents do not prescribe which resolution is right in any given case. They commit to surfacing the variance so that the resolution conversation has data behind it.

The data point as of the date of this charter: in the seven weeks since the deep-dive program began producing output, one procedure has reached production (`lsp_SrtGetShipToteIfExists`) and one is currently under review (`lsp_RxfGetListOfManualFillGroups`). The cataloged backlog at our end is materially larger than that, and grows every week.

This pace creates two practical consequences for the program, and the rest of the charter is designed around them.

The first consequence is that the value of any single analysis is realized only after the recommendation has flowed through the full pipeline. Until that point, the analysis is potential value rather than realized value. The program plans accordingly: we measure ourselves against the rate at which we publish high-quality, deployable recommendations. The fleet-wide read reduction figures are tracked separately, and the gap between cataloged-and-ready and deployed-to-production is itself one of the metrics in section 7.

The second consequence is that there is also a steady stream of outstanding information requests from our side to the iA team. These are typically questions about application contract, about the conditions under which a procedure is called, or about whether a recommendation is consistent with iA's longer plan for an area of the codebase. The turnaround on these requests has been measured in weeks rather than days, presumably because the answers route through the same iA development teams that own the validation step. Where we do not have an answer, we proceed with the analysis using the most defensible assumption and document the assumption explicitly so that the iA reviewers can correct it during their review. Where an answer is essential before we can proceed, we record the question against the relevant analysis and continue with the next item in the backlog.

---

4. Program Streams

The program comprises three streams of work. Each stream has a dedicated plan document. This charter describes the relationship between them and the cadence at which they will run.

4.1 Query Store Triage and Stored Procedure Refactor Program

The first stream is the deep-dive refactor program for the most expensive procedures in the database. It is the source of the most visible recommendations, and it is the program that will

continue to consume the largest share of my time over the coming quarter.

The full plan, methodology, and procedure-by-procedure status is in the Query Store Triage and Refactor Program plan. The short version: the MFC DBA team and I have stood up a diagnostic toolkit, ten read-only scripts that profile a site in under two minutes, and a refactor workflow that produces a self-contained analysis package per procedure with an apples-to-apples evidence comparison of the original and the refactored body. The pilot has been completed against `lsp_ShpGetOrdersForTopReadyToShipGroup` (Row 15 of the MFC database optimization tracking sheet), and analysis scaffolding is in place for Rows 16 through 21. Rows 22 through 39 are queued.

Each completed analysis is published as a recommendation to the iA DBA team. Production deployment is the iA team's call and runs on the iA team's cadence. The triage plan describes the recommendation handoff and the deployment-status tracking that complements the cataloged backlog.

4.2 Maintenance Operations Plan

The second stream addresses the nightly maintenance suite. The full plan is in the Maintenance Operations Plan. The short version: I have completed an end-to-end assessment of the orchestrating nightly maintenance procedure and the nineteen child procedures it calls, identified critical bugs that are silently preventing some purges from running, and laid out a sequence of corrective recommendations. Several of the recommendations have already been written up as discrete change proposals. The remainder are queued in the backlog.

The maintenance plan has a near-term shape (recommend the bug fixes, recommend the unbounded-loop retrofits, recommend the wide-column read narrowing, recommend the legacy temp-table pattern modernization) and a long-term shape (recommend replacing the iterative block deletes on the highest-volume tables with partition-switch operations once partitioning is in place). The bridge between near-term and long-term is the partitioning effort in stream three.

Expected outcome: the maintenance window for the highest-volume tables drops from multi-hour to seconds once partition switches are wired in. Before that, the in-place fixes restore correct behavior on procedures that are silently no-ops today and prevent transaction log inflation events. Realization of the outcome depends on the iA team's deployment cadence in the same way that the refactor program does.

4.3 Scalability Roadmap and Partitioning

The third stream is the long arc. Even if every recommendation in the backlog is accepted and deployed, the underlying architecture has to evolve to handle the next several years of growth. The plan is in the Scalability Roadmap.

The current proof of concept is a five-module body of work that covers what partitioning is, the migration approach, the indexing strategy, the switch operations that integrate with maintenance, and the monitoring that confirms the partitioning is doing what we expect. The POC is complete. Graduating it from a proof of concept to a production rollout requires the same iA-side review, validation, and deployment that any other database change requires, plus coordination with the application teams whose code accesses the partitioning candidate tables.

Beyond partitioning, the roadmap also covers a small number of architectural changes that surfaced during the refactor work and are too large for a single procedure scope. The most prominent is replacing the polling pattern that drives `lsp_PmssIwebGetTopQueuedTrx` (currently running 825 million times per month fleet wide) with a Service Broker queue or signal-based handoff. Several other items are at the same shape and are itemized in the roadmap document. All of them are recommendations rather than committed work, since they require iA development team capacity that we do not control.

---

5. Methodology

The program operates under four rules that are not negotiable on our side of the pipeline.

The first rule is that every recommendation is grounded in evidence. Every refactor produces a side-by-side comparison of the original and refactored procedure under the same data state, on a warm cache, with a result set that has rows. Conclusions drawn from cold-cache numbers, zero-row test runs, or comparisons across different data states do not count, and the workflow document is explicit on this point. The evidence package is part of the deliverable to the iA reviewers, not an internal artifact, because the reviewers need it to do their own validation.

The second rule is that every recommendation produces durable knowledge alongside the proposed code change. Each procedure under review has a self-contained analysis package containing the original body, the refactored body, and a written analysis that follows an eleven-section template (surface area, performance overview, evidence of original, issue identification, first principles, refactor commentary, risk and rollback, evidence of refactor, comparison and verdict, validation checklist, and open items). The first principles section cites the relevant entry from the masterclass library of pattern notes. The library is written in plain prose. When a refactor surfaces a principle that no existing entry covers, a new note is written.

The third rule is that lessons learned from corrections are codified immediately in a running lessons-learned log and applied to subsequent work. The cost of writing the rule down is small. The cost of repeating the same mistake on the next procedure is large. Several of the lessons currently logged came directly from issues raised by the iA reviewers during their own validation work. Those lessons are now part of the workflow and prevent the same issue from recurring on subsequent submissions.

The fourth rule is that no refactor is closed at production deployment. A refactor that has shipped is a refactor still under observation. The MFC DBA team and I monitor the deployed procedure at every site for a defined post-deployment window and compare the actual behavior against the pre-deployment baseline at each site. Where a site shows improvement consistent with the prediction, the change is closed at that site. Where a site shows no improvement or shows degradation, the change is not closed there. We open a per-site root cause analysis, document the data-shape or access-pattern explanation if one exists, and either propose a follow-up adjustment or accept the variance with an explicit explanation. The post-deployment monitoring is part of the methodology, not an extra. A refactor without post-deployment evidence is a refactor whose outcome has not been confirmed at scale.

These rules have already paid off. The Row 15 pilot uncovered an OPTION clause translation issue that would have shipped a parse error to production if the workflow had not insisted on diffing the tested body against the committed body before declaring done. That experience is now captured in the lessons log and in the workflow document so that no subsequent refactor can hit the same trap.

---

6. Phasing and Cadence

The program runs in three overlapping phases. The phases describe the cataloging and recommendation work that the MFC DBA team and I produce. Production deployment of any recommendation runs on a separate cadence set by the iA team, and the phasing below does not commit the iA team to a deployment schedule.

Phase 1 (Q2 2026): Triage and Catalog. The active phase. The refactor program runs through Rows 15 through 39 of the tracking sheet. The maintenance bug-fix recommendations (the SmartShelf delete loops, the double-delete pattern, the unbounded loops) are published as discrete change proposals. The diagnostic toolkit gets exercised at every site so we have a current cross-MFC view at all times. Phase 1 delivers the bulk of the near-term recommendation backlog. The deployment of any specific recommendation in this phase is the iA team's decision.

Phase 2 (Q3 2026): Recommendation Depth and Partitioning Pilot Recommendation. With the worst offenders cataloged, the focus shifts to maintenance modernization recommendations that do not strictly require partitioning (legacy temp-table check substitution, modernized block-selector patterns, persisted work tables across loop iterations) and to the recommendation for the partitioning rollout pilot table. The recommendation includes the candidate table choice, the migration plan for that table, and the proposed monitoring during the pilot window. Whether the iA team and the application teams choose to execute the pilot in Phase 2 or later is their decision.

Phase 3 (Q4 2026 through Q1 2027): Architectural Recommendations and Backlog Maturation. With the procedure-level recommendations cataloged, the focus shifts to the architectural items: the Service Broker conversion of the PMSS polling pattern, the LOB strategy for the image tables, the read workload separation evaluation, and the partition-switch integration recommendations for the maintenance procedures. Each one is published as a recommendation with the supporting analysis. By the end of Phase 3 the cataloged backlog should cover everything we have currently identified, organized in a form that the iA team can prioritize against their own roadmap.

The phasing is realistic, not aspirational. Each phase has a small number of explicit gates that have to be passed before the next phase begins. The gates are enumerated in the individual stream documents.

---

7. Expected Outcomes and Measurement

I will measure the program against five metrics and report against them at a regular cadence. The first three measure realized outcomes in production and are therefore gated by the iA team's deployment cadence. The last two measure the rate and quality of the cataloging work itself, which is what we directly control.

The first metric is cross-MFC total reads per month for the top fifty offenders. The Query Store capture is reproducible and the methodology is documented. The May 7, 2026 capture serves as the baseline. I will publish a fresh capture quarterly and report the delta against baseline by procedure and at the program level. The delta moves only when deployed recommendations land at the sites, so this metric is a downstream signal.

The second metric is the duration of the nightly maintenance window at each site, broken down by procedure. The current nightly run is observable through the existing logging infrastructure. The improvement target is straightforward: keep the window from growing as data accumulates, and reduce it materially once partition switches are deployed.

The third metric is plan stability. The cross-MFC Query Store evidence shows that several procedures are producing multiple plan variants for the same statement at the same site, with up to a 70,000-fold spread in reads per execution between variants. Plan stability is a leading indicator of parameter sniffing and index coverage problems. I will track the count of statements producing more than one plan variant in the top fifty per site and target a steady reduction.

The fourth metric is the publishing rate of the cataloged backlog: the count of analyses completed and published as deployable recommendations per quarter, with each one carrying a complete evidence package. This is the metric that reflects the work of the MFC DBA team and me.

The fifth metric is the deployment-cadence gap: the count and the age of recommendations sitting in the backlog versus the count deployed to production. This is the metric that exposes the gap between the analytical pace and the deployment pace. Reporting on this metric is not intended as criticism of the iA team's process. It is intended to give engineering leadership and the iA team a shared view of the backlog so that prioritization conversations have data behind them.

The sixth metric is the post-deployment per-MFC variance: for every refactor that has been deployed, the count of sites where the refactor delivered the predicted improvement, the count of sites where the refactor showed no measurable change, and the count of sites where the refactor degraded performance compared to the pre-deployment baseline at that site. The third bucket is the one that triggers a per-site root cause analysis. Tracking the three counts over time gives leadership a direct read on whether uniform code is producing uniform results across the fleet, and on the cumulative outcome of the deployments that have already happened.

Beyond the metrics, the program produces durable knowledge as a deliverable in its own right. The masterclass library, the per-procedure analyses, the lessons-learned log, and the diagnostic toolkit are all artifacts that any engineer joining either the MFC DBA team or the iA team can read and act on without my involvement. The intent is that by the end of Phase 3 the working knowledge of how to do this kind of optimization is held by both DBA teams jointly, not held by a single consultant who is a single point of failure.

---

8. Risks and Dependencies

The largest risk in the program is the deployment cadence gap. As of this charter, one procedure has reached production and one is in review across a seven-week window. The cataloged backlog is materially larger than that, and the procedures on the list are continuing to produce cancellations and performance failures in production while they wait. If the cadence does not increase, the program will not keep up with the rate at which the underlying conditions are degrading, and the operational events will continue to occur on procedures we have already analyzed and recommended. The mitigation is two-sided. On our side, we are doing everything we can to make each recommendation as easy as possible to review and accept, including the complete evidence package, the explicit risk and rollback section, and the validation checklist. On the iA side, the mitigation requires conversation about how to increase throughput without compromising the standards that the multi-stage review is designed to protect.

The second risk is the information request turnaround. Several analyses have outstanding questions that we have submitted to the iA team and that have not yet returned answers. The current turnaround on these requests is measured in weeks. Where an answer is not strictly blocking, we proceed with a defensible assumption and document it. Where an answer is

blocking, the analysis sits incomplete in the backlog. The mitigation here is a shared-document mechanism for these questions so that they are visible to the right person on the iA side and so that the cost of an unanswered question is visible to leadership.

The third risk is that we underinvest in the validation harness for the recommendations. A recommendation that ships with thin evidence is a recommendation that may regress under a different data shape or load profile at a different site. The workflow forbids this, but the workflow only works if the people doing analyses actually follow it. The mitigation is the apples-to-apples evidence protocol embedded in the workflow document, which lists explicit pass and fail criteria.

The fourth risk is that the partitioning rollout, when it eventually executes, is more disruptive than planned. Partitioning a busy production table is not a trivial migration and the POC was deliberately scoped to controlled conditions. The mitigation is the staged rollout described in the Scalability Roadmap, the migration approach in the POC, and the rollback path that the migration approach preserves.

The fifth risk is environmental. Several of the diagnostic and refactor activities depend on Query Store being enabled and configured correctly at every site. Where Query Store is not enabled, the diagnostic toolkit falls back to the wait stats and index usage scripts, which are useful but less precise. Confirming Query Store status at every site early in Phase 1 is on my immediate to-do list.

The sixth risk is the tension between uniform code and non-uniform data. The cross-MFC Query Store evidence already shows that the same statement at different sites can vary by orders of magnitude in reads per execution. That variance is data-shape variance and access-pattern variance, not code variance. A refactor that is optimal against one data shape may not be optimal against all data shapes in the fleet. The iA team's preference for a uniform production codebase is a legitimate engineering value, but uniform code interacting with non-uniform data cannot be assumed to produce uniform outcomes. The mitigation is the post-deployment per-MFC monitoring described in the methodology and metrics sections. When variance shows up after deployment, it is investigated, root-caused, and either explained or addressed. The program does not commit in advance to which resolution is right in any given case. It commits to ensuring that when the variance is real, it is visible.

---

9. Roles

The program is a collaboration across three working parties.

I am leading the program from the analysis and recommendation side. I am the author of the analyses, the refactors, the diagnostic toolkit, the masterclass library entries, and the planning

documents. I am the single point of accountability for the methodology and for the quality of the deliverables on our side of the pipeline.

The MFC DBA team works alongside me on the analysis side. They run the diagnostic toolkit at each site, gather the cross-MFC evidence, contribute to the per-procedure analyses, and surface the operational context (the cancellations and performance failures observed in production) that informs the prioritization. They are co-authors of the backlog, not consumers of it.

The iA DBA team owns the first review gate. Every recommendation we publish flows to the iA DBA team for review against their standards for the production codebase. Their review is what determines whether a recommendation moves to the next stage of the pipeline.

The iA software development teams own the second review gate. After the iA DBA review, the recommendation goes to the iA developers responsible for the affected functional area for validation. They confirm that the change preserves application contract and that there are no side effects in calling code paths.

The iA operations team owns the non-production testing window and the production deployment.

Engineering leadership owns prioritization between the cataloged backlog and the architectural backlog when they compete, and the funding for the longer-arc work in Phase 3.

The deliberate split is that the MFC DBA team and I produce the recommendations and the supporting evidence; the iA team controls what flows from recommendation to production deployment. Both halves are essential to the program. Neither half can deliver the program-level outcomes alone.

---

10. Backlog Strategy

Given the deployment cadence reality, the practical work product of the program is a documented, evidence-backed backlog of recommendations. The backlog is in two views.

The first view is the procedure-level cohort. The MFC database optimization tracking sheet rows 15 through 39 are the deep-dive cohort. Each row in the cohort eventually has a self-contained analysis package containing the pre-refactor body, the proposed refactored body, and the written analysis. The tracking sheet itself carries the deployment status (cataloged, in iA review, in non-prod testing, in production) so that the gap between cataloged and deployed is visible at a glance.

The second view is the architectural backlog. Items that are larger than a single procedure (Service Broker conversion, LOB strategy, filtered indexes on hot predicates, partition-switch integration in maintenance) live in the Scalability Roadmap. Each item carries a recommendation and the supporting analysis, in a form that the iA team can prioritize against their own roadmap.

The backlog is organized so that any recommendation in it is shovel-ready when the iA team has bandwidth to pick it up. The evidence package is complete, the risk and rollback section is explicit, the validation checklist is in place, and the supporting reference material is available. The intent is to make the cost of accepting a recommendation as low as possible on the iA side, so that throughput can grow when the iA team's bandwidth allows it.

---

11. How to Read the Rest of the Documentation

This charter is the entry point. From here:

- The Query Store Triage and Refactor Program plan covers the immediate refactor program in depth, including the deployment status of each procedure in the cohort.
- The Maintenance Operations Plan covers the nightly maintenance assessment, the bug-fix recommendations, and the maintenance modernization plan.
- The Scalability Roadmap covers the partitioning rollout recommendation and the longer arc architectural items.

For procedure-level depth, every procedure under analysis has its own analysis package with the full methodology applied. For pattern-level depth, the masterclass library is the canonical reference.

I will keep these four documents current. Any change in scope, schedule, or methodology lands first in the appropriate document and is reflected back here in the next quarterly update.